

Name of the Student	N.AKSHAYA
Reg. No	192425129
GitHub Link	https://github.com/akshaya24032007/MLA0405-DeepLearning/tree/main/Assessment/CO3

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

Game Based Learning

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0405

SLOT :D

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Duration: 1 Hour

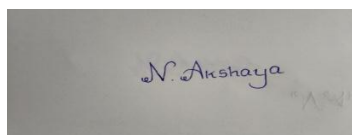
Assessment Tool Weightage: 35%

Marks: 50

Code of Conduct:

I N.AKSHAYA(192425129) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student:N.AKSHAYA

Criteria	Understanding of Concept (2.5)	Conceptual Clarity (2.5)	Problem-Solving Ability (2)	Solution Design & Application (2)	Teamwork & Game Participation (1)	Total (10)
Weightage	25 %	25%	20%	20 %	10 %	100%
Round 1						
Round 2						
Total						

Signature of the Faculty:

Total Marks :

Question:

Design and participate in a two-round game-based learning activity to explore the fundamental concepts of Convolutional Neural Networks (CNNs), including architecture, layers, filters, parameter sharing, regularization, AlexNet, ResNet, and real-world applications. Progress through five levels in each round, applying your knowledge to solve challenges involving CNN design, feature extraction, architecture comparison, and practical applications.

The Game Progression

Round 1:

Basic CNN Concepts → Filters → Parameter Sharing → Architecture → AlexNet vs ResNet

Round 2:

Convolution → Feature Maps → Regularization → Architecture Comparison → Real-World Application

ROUND 1: CNN Architecture Builder (Learn by constructing and understanding CNN architectures layer by layer)

Level	Challenge	Description
Level 1	CNN Layer Matcher	Match CNN layers such as Convolution, Pooling, Flatten, and Fully Connected layers with their respective functions and roles in the network.
Level 2	Filter Finder	Identify suitable filters and kernels for feature extraction and understand how filter size, stride, and padding affect the output feature map.
Level 3	Parameter Sharing Puzzle	Determine how the same filter weights are reused across different image locations and calculate the number of parameters in a convolutional layer.
Level 4	CNN Architecture Debugger	Identify and correct problems in a CNN architecture, such as incorrect layer order, incompatible dimensions, and inappropriate layer selection.
Level 5	AlexNet vs ResNet Challenge	Compare AlexNet and ResNet architectures and identify how their architectural approaches differ, particularly the use of residual connections in ResNet.

ROUND 1 – CNN ARCHITECTURE BUILDER

Level 1 – CNN Layer Matcher

Answer:

A CNN consists of different layers, where each layer performs a specific task.

- **Convolution Layer:** Applies learnable filters to the input image and extracts important features such as edges, corners, textures, and patterns.
- **Pooling Layer:** Reduces the spatial dimensions of feature maps while retaining important information. Max pooling is commonly used.
- **Flatten Layer:** Converts the 2D/3D feature maps into a one-dimensional vector.
- **Fully Connected Layer:** Uses the extracted features to perform classification and generate the final prediction.

The layers must be arranged correctly so that features are gradually extracted and finally classified.

Level 2 – Filter Finder

Answer:

Filters or kernels are small matrices used to extract features from an image. A **3×3 kernel** can detect edges, textures, and simple patterns.

- **Filter size:** Determines the region of the image examined at a time.
- **Stride:** Determines how many pixels the filter moves during convolution.
- **Padding:** Adds extra pixels around the image and helps control the output feature-map size.

For example, an edge-detection kernel can highlight sudden changes in pixel intensity and produce an edge feature map.

The filter size, stride, and padding should be selected according to the required feature extraction and output dimensions.

Level 3 – Parameter Sharing Puzzle

Answer:

Parameter sharing is an important property of CNNs. The **same filter weights are reused across different locations of an image**. Therefore, CNNs do not need separate weights for every image position.

For a convolutional layer:

Number of Parameters = (Filter Height × Filter Width × Input Channels + Bias) × Number of Filters

For example, for a **3×3 filter, 3 input channels, and 16 filters**:

Parameters = (3 × 3 × 3 + 1) × 16 = 448

Parameter sharing reduces computational complexity and allows the CNN to detect the same feature at different locations.

Level 4 – CNN Architecture Debugger

Answer:

A CNN architecture should follow a logical sequence:

Input → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Flatten → Fully Connected → Output

Possible architecture problems include:

1. Incorrect layer order.
2. Incompatible input and output dimensions.
3. Incorrect number of channels.
4. Unnecessary or inappropriate layers.
5. Incorrect connection between feature extraction and classification layers.

Solution: Check the dimensions after every convolution and pooling layer and ensure that the Flatten layer receives a valid feature map before classification.

This debugging process ensures that the CNN architecture is valid and computationally efficient.

Level 5 – AlexNet vs ResNet Challenge

Answer:

AlexNet and **ResNet** are important CNN architectures, but they use different architectural approaches.

Feature	AlexNet	ResNet
Architecture	Conventional deep CNN	Residual CNN
Main idea	Deep convolutional layers	Residual/skip connections
Training	More difficult as depth increases	Easier for very deep networks
Key advantage	Strong CNN performance	Handles very deep architectures effectively

ResNet uses residual connections, which allow information and gradients to pass through shortcut paths. This helps reduce the difficulty of training deep networks.

Conclusion: ResNet is generally more suitable for complex problems requiring a very deep CNN.

ROUND 2: CNN Feature Detective (Learn by exploring how CNNs extract features, avoid overfitting, and solve real-world problems)

Level	Challenge	Description
Level 1	Edge Detection Explorer	Apply simple (3x3) filters to an image and determine how convolution operations can detect edges and other basic features.
Level 2	Feature Map Detective	Analyze feature maps produced by different CNN layers and identify the progression from simple features such as edges to complex patterns and objects.
Level 3	Regularization Rescuer	Identify overfitting in a CNN model and select suitable regularization techniques to improve generalization and model performance.
Level 4	CNN Architecture Race	Compare ResNet and AlexNet for different image-classification applications and determine which architecture is more suitable for a given problem.
Level 5	CNN Application Master	Design a CNN-based solution for a real-world application such as medical image analysis, face recognition, object detection, or autonomous driving, and justify the selected architecture and layers.

ROUND 2 – CNN FEATURE DETECTIVE

Level 1 – Edge Detection Explorer

Answer:

Convolution can be used to identify edges by applying a small kernel to an image. A **3×3 edge-detection filter** examines neighboring pixels and responds strongly where there is a sudden change in intensity.

The convolution process is:

Image → Apply 3×3 Filter → Calculate Weighted Sum → Generate Feature Map

Strong values in the resulting feature map indicate possible edges.

This demonstrates how CNNs initially extract simple visual features before learning more complex patterns.

Level 2 – Feature Map Detective

Answer:

Feature maps show the features detected by different CNN layers.

- **Early layers:** Detect edges, lines, and simple textures.
- **Middle layers:** Detect shapes, curves, and combinations of basic patterns.
- **Deep layers:** Detect complex patterns, object parts, and meaningful objects.

Thus, CNN feature extraction progresses from **simple features to complex features**.

Example:

Edges → Textures → Shapes → Object Parts → Complete Object

This hierarchical feature extraction enables CNNs to perform image classification effectively.

Level 3 – Regularization Rescuer

Answer:

Overfitting occurs when a CNN performs very well on training data but performs poorly on unseen validation or test data. It means the model has learned the training examples too closely instead of learning general patterns.

Suitable regularization techniques are:

- **Dropout:** Randomly disables neurons during training.
- **L2 Regularization:** Penalizes large model weights.
- **Data Augmentation:** Creates variations of training images.
- **Early Stopping:** Stops training when validation performance begins to decrease.

Best solution: Combine data augmentation with dropout and monitor validation performance. These methods improve the model's ability to generalize to new images.

Level 4 – CNN Architecture Race

Answer:

The choice between AlexNet and ResNet depends on the application.

AlexNet: Suitable for relatively simpler image-classification tasks and provides a basic deep CNN architecture.

ResNet: More suitable for complex image-classification problems because it can use many layers with residual connections.

Decision: For a large and complex image dataset requiring deep feature learning, **ResNet is preferred.**

Reason: Residual connections improve information flow and make training deeper CNN architectures more effective.

Level 5 – CNN Application Master

Answer:

Application: Medical Image Analysis

CNNs can be used to classify medical images such as X-rays, CT scans, or MRI images.

Proposed Architecture:

Input Medical Image → Convolution → ReLU → Pooling → Residual Blocks → Fully Connected Layer → Softmax Output

Working:

1. The input image is provided to the CNN.
2. Convolution layers extract important visual features.
3. Pooling reduces feature-map dimensions.
4. Residual blocks learn deeper and more complex features.
5. Fully connected layers perform classification.
6. Softmax provides the final class probabilities.

Architecture Selection: ResNet is selected because residual connections support deep feature extraction and effective training.

Expected Outcome: The system can classify medical images into relevant categories and support computer-aided analysis.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Conceptual Clarity	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Problem-Solving Ability	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design & Application	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Teamwork & Game Participation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand